SIMATS ENGINEERING

Engineer to Excel

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

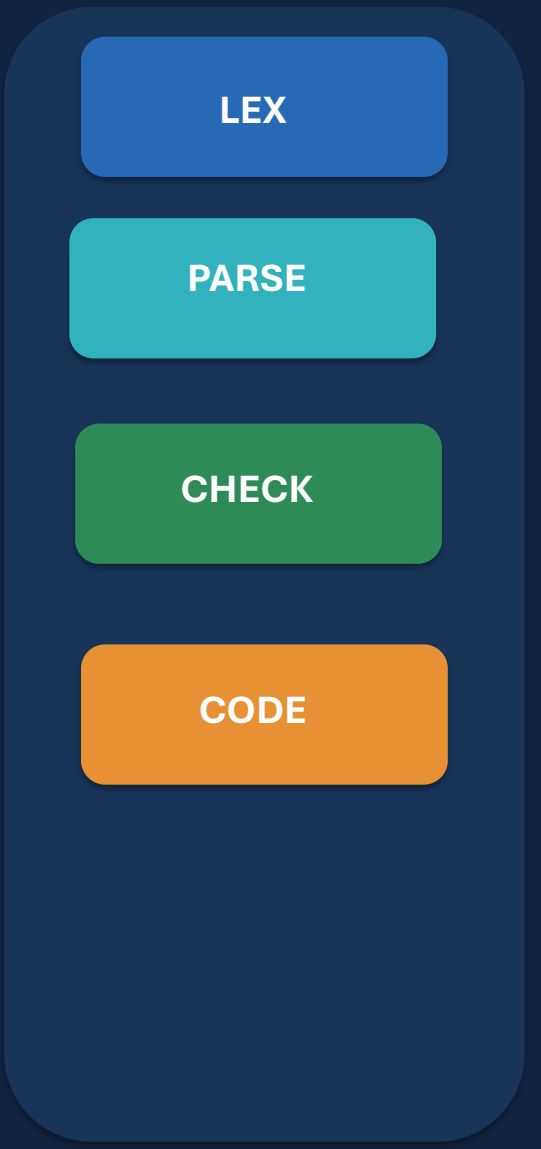
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PHASES OF A COMPILER

TECHNICAL PRESENTATION

From source code to executable machine code

CH.V.Ramayogi-192424319



Compiler: The Big Picture

A compiler translates a high-level program into target code through a sequence of analysis and synthesis phases.

Example source statement

C SOURCE

```
int total = 2 + 3;
```

1. **Lexical analysis**
   Characters → tokens
2. **Syntax analysis**
   Tokens → parse tree
3. **Semantic analysis**
   Meaning and types
4. **Intermediate code**
   Machine-independent form
5. **Optimization**
   Improve efficiency
6. **Code generation**
   Target instructions

Why phases?

- Each phase has a specific responsibility
- Errors can be detected at the right stage
- Intermediate forms support optimization
- The target code depends on the machine

1. Lexical Analysis

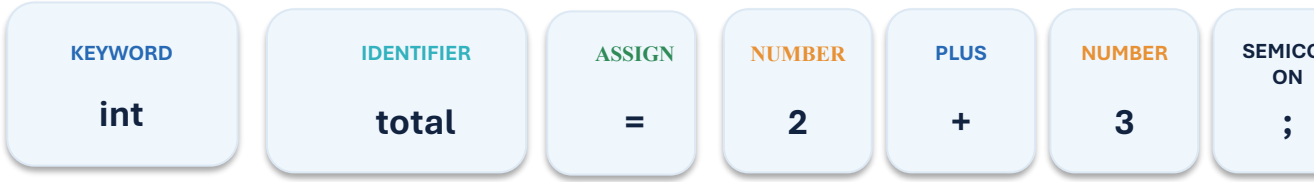
The scanner reads characters and groups them into tokens.

- First phase of compilation
- Recognizes keywords, identifiers, operators and constants
- Skips spaces, tabs and comments
- Uses patterns that can be implemented with finite automata
- Reports invalid character sequences

INPUT

```
int total = 2 + 3;
```

Token stream



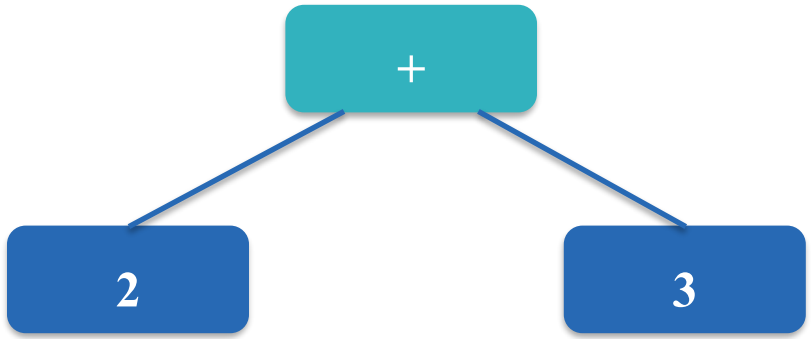
Output example Tokens preserve the program's meaningful units; whitespace is not normally passed on.

2. Syntax Analysis (Parsing)

The parser checks whether the token sequence follows the language grammar.

- Checks grammatical structure using context-free grammar (CFG)
- Builds a parse tree or abstract syntax tree (AST)
- Detects missing operators, parentheses or semicolons
- Common parser families include LL and LR

Simplified expression tree for 2 + 3



VALID INPUT

```
total = 2 + 3;
```

Error example: `total = 2 + ;`

3. Semantic Analysis

The compiler checks whether a syntactically correct program also makes sense.

- Checks data types and type compatibility
- Verifies declarations and variable scope
- Uses the symbol table to track identifiers
- Reports errors such as undeclared variables or invalid operations
- Annotates the syntax tree for later code generation

VALID EXAMPLE

```
int age = 20;
```

✓ **Declared variable + compatible type**

SEMANTIC ERROR

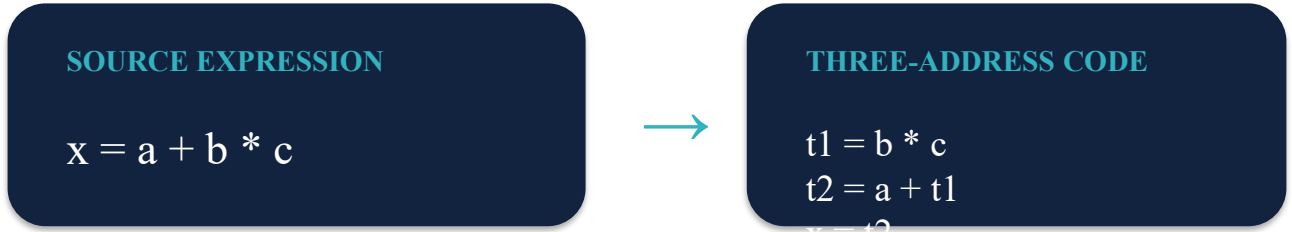
```
int age = "twenty";
```

In a strongly typed language, a string cannot initialize an int.

4. Intermediate Code Generation

Produces a machine-independent representation between analysis and target-code generation.

- Converts the checked program into an intermediate representation (IR)
- Three-address code is a common example
- Makes optimization easier
- Separates language-specific front-end from machine-specific back-end



Each instruction performs a small, explicit operation.

Benefit: the same IR can target different processors.

5. Code Optimization

Improves the intermediate code without changing the program's intended result.

- Removes redundant computations
- Simplifies constant expressions
- Can reduce execution time and code size
- Uses techniques such as dead-code elimination and loop optimization

BEFORE

```
x = 4 * 5;
y = x + 0;
```

→

AFTER

```
x = 20;
y = x;
```

Example techniques

Constant folding • dead-code removal • loop optimization

Optimization must preserve program behavior.

6. Code Generation

The back end maps the optimized representation to target instructions.

- Produces assembly or machine code for a target architecture
- Assigns values to registers and memory locations
- Chooses suitable machine instructions
- Considers processor and platform constraints

INTERMEDIATE CODE

t1 = 2 + 3

→

ILLUSTRATIVE ASSEMBLY

MOV R1, #2
ADD R1, #3

The exact instructions depend on the target CPU.

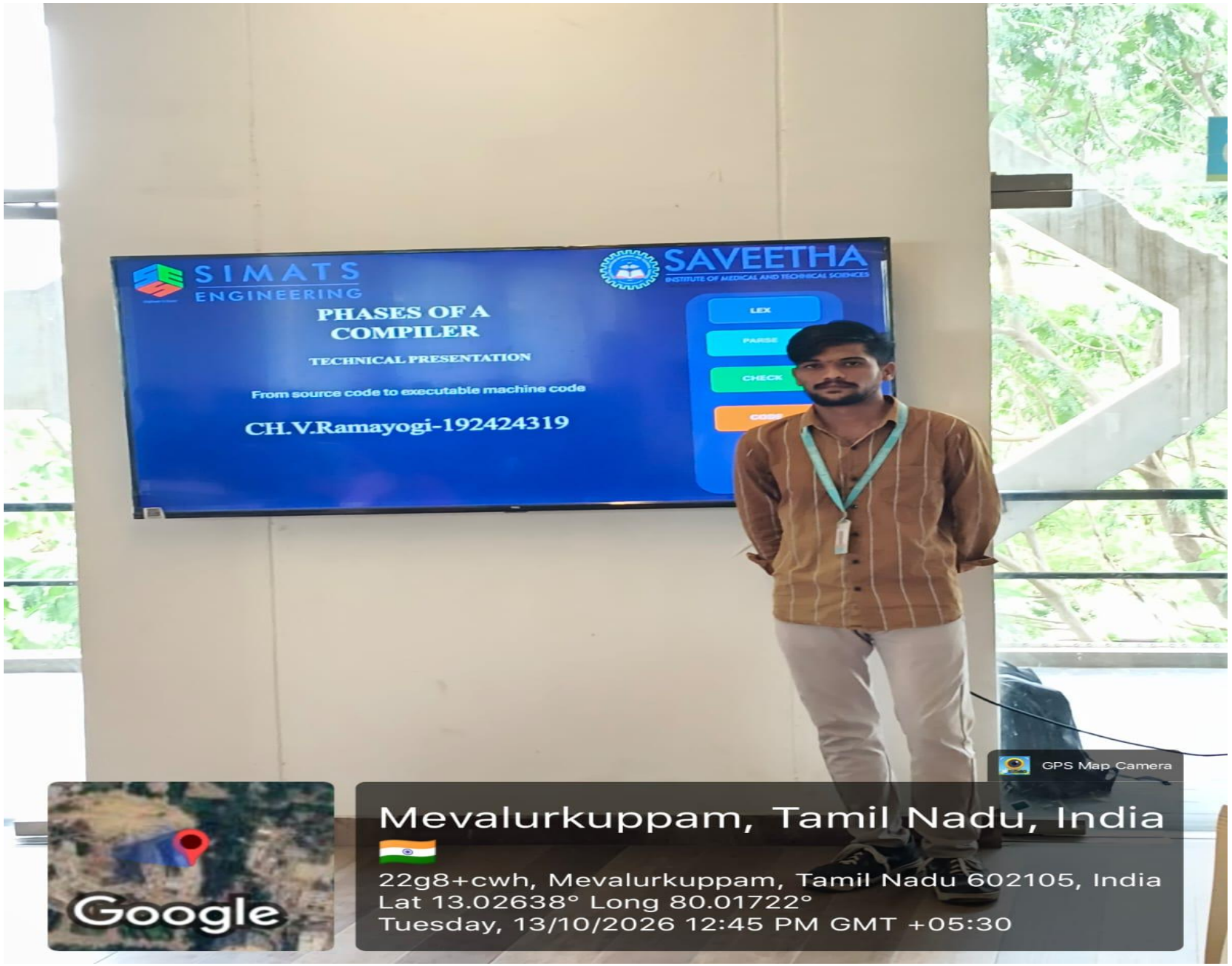
Final result: object code / executable after linking.

One Program, Six Transformations

Example: int total = 2 + 3;

Source	int total = 2 + 3;
Tokens	int • total • = • 2 • + • 3 • ;
Syntax	declaration / expression tree
IR	t1 = 2 + 3; total = t1
Optimized	total = 5
Target	**CPU-specific instructions**

Key takeaway: each phase transforms the program and checks a different kind of correctness.


SIMATS
ENGINEERING
SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
PHASES OF A
COMPILER
TECHNICAL PRESENTATION
From source code to executable machine code
CH.V.Ramayogi-192424319
LEX
PARSE
CHECK
GPS Map Camera
Google
Mevalurkuppam, Tamil Nadu, India
22g8+cwh, Mevalurkuppam, Tamil Nadu 602105, India
Lat 13.02638° Long 80.01722°
Tuesday, 13/10/2026 12:45 PM GMT +05:30